

Meet in the Middle (MITM) - Interview Notes

When should I think of Meet in the Middle?

Meet in the Middle is **not a DP pattern**. It is a separate algorithmic technique used when brute force is too expensive, but the input size is small enough to split into two halves.

Typical clues:

- $n \leq 40$
- Subset / Partition / Subsequence problems
- Brute force is $O(2^n)$
- Element values (or total sum) are very large, making Knapsack DP impractical

DP vs Meet in the Middle

Both are often used for **subset/partition** problems.

The deciding factor is **the constraints**, not the problem statement.

Use Knapsack DP when:

- Total sum is reasonably small.
- DP state can be built over the sum.

Example:

```
n = 30
nums[i] ≤ 100

Maximum sum = 3000
```

DP state:

```
dp[sum]
```

Time:

```
 $O(n \times \text{sum})$ 
```

Examples:

- LeetCode 416 - Partition Equal Subset Sum

- LeetCode 1049 - Last Stone Weight II
- Minimum Subset Sum Difference

Use Meet in the Middle when:

- n is small (typically ≤ 40)
- Element values (or sums) are huge
- DP over sums would be too large

Example:

```
n = 40
nums[i] ≤ 107

Maximum sum ≈ 4 × 108
```

A DP array of hundreds of millions (or billions with negatives) is not feasible.

Instead, split the array into two halves.

Examples:

- LeetCode 1755 - Closest Subsequence Sum
- LeetCode 2035 - Partition Array Into Two Arrays to Minimize Sum Difference

Meet in the Middle Idea

Instead of exploring

```
2n
```

subsets,

split the array into two halves.

```
nums
      |
-----
|           |
Left       Right
```

Generate all subset sums of each half.

Left subset sums
Right subset sums

Each half has only

$2^{(n/2)}$

subsets.

Sort one list.

For every subset sum in the other list, use Binary Search to find the best matching subset sum.

This reduces

$O(2^n)$

to roughly

$O(2^{(n/2)})$

Generic MITM Algorithm

Split array into two halves

Generate all subset sums of left half

Generate all subset sums of right half

Sort right subset sums

For each left subset sum:

 target = ... (problem dependent)

 Binary Search on right subset sums

 Update answer

Generating Subset Sums

Use standard Pick / Don't Pick recursion.

```
generate(index, sum):  
  
    if index == nums.length:  
        add sum  
        return  
  
    // Don't Pick  
    generate(index + 1, sum)  
  
    // Pick  
    generate(index + 1, sum + nums[index])
```

Closest Subsequence Sum (LeetCode 1755)

Problem:

Choose **any subset** whose sum is closest to `goal`.

Return:

```
|goal - subsetSum|
```

Algorithm:

```
Split array  
  
Generate left subset sums  
  
Generate right subset sums  
  
Sort right subset sums  
  
For every leftSum:  
  
    need = goal - leftSum  
  
    Binary Search right subset sums  
  
    Check insertion point and previous value
```

Update minimum difference

Return the minimum absolute difference.

Minimum Subset Sum Difference / Last Stone Weight II

These are classic **0/1 Knapsack** problems because the total sum is small.

DP state:

dp[sum]

Meaning:

Can we form this sum?

Find the largest achievable subset sum $\leq \text{totalSum} / 2$.

Answer:

$(\text{totalSum} - \text{subsetSum}) - \text{subsetSum}$

Last Stone Weight II is NOT a Heap Problem

A heap is useful only when the problem forces us to repeatedly choose the **two heaviest** stones.

Last Stone Weight II does **not** say that.

It allows choosing **any two stones** at every step.

Therefore:

- There are many possible sequences of smash operations.
- A greedy heap explores only one sequence.
- The optimal sequence may be different.

The problem reduces to partitioning the stones into two subsets whose sums are as close as possible, which is exactly a Knapsack DP problem.

Interview Rule of Thumb

Subset / Partition problem

```
|
+--> Sum is small?
|      |
|      +--> Knapsack DP
|
+--> Sum is huge but n <= 40?
      |
      +--> Meet in the Middle
```

The algorithm is chosen by the **constraints**, not by the wording of the problem.